
Sage: An Offline-First Academic Assistant Using RAG with Tool-Integrated Workflows

Ahmad Raza

ahmad.raza@scholar.tu.edu.pk

Abdullah Khan

abdullah.khan@scholar.tu.edu.pk

Supervisor: N/A

Date: **03 June 2026**

An FYP Report submitted for the degree of
Bachelor of Science in Software Engineering (2022–2026)

**Department of Computer Science & Information Technology
Thal University Bhakkar**

Acknowledgements

The authors express their sincere gratitude to [Supervisor Name] for their continued guidance and constructive feedback throughout the course of this project. Appreciation is also extended to the faculty of the Department of Computer Science & Information Technology at Thal University Bhakkar for providing the academic foundation that made this work possible. Finally, the authors acknowledge the open-source communities behind LangChain, LangGraph, llama.cpp, ChromaDB, FastEmbed, and the broader ecosystem of tools upon which Sage is built.

Contents

List of Abbreviations	5
Chapter 1: Introduction	7
1.1 Background	7
1.2 Problem Statement	7
1.3 Motivation	7
1.4 Project Objectives	7
1.5 Report Organisation	8
Chapter 2: Literature Review	8
2.1 Overview of the Domain	8
2.2 Related Work and Existing Systems	9
2.3 Identified Gaps	9
Chapter 3: Requirements Analysis	9
3.1 Stakeholder Analysis	9
3.2 Functional Requirements	10
3.3 Non-Functional Requirements	11
3.4 Use Case Descriptions	11
3.4.1 Ingest Academic Document	11
3.4.2 Ask an Explain Question	11
3.4.3 Generate and Take a Quiz	12
3.4.4 Conduct Multi-Source Research	12
3.4.5 Generate a Study Plan	13
Chapter 4: System Design	13
4.1 System Architecture Overview	13
4.2 Component Breakdown	15
Router (<code>agents/router.py</code>)	15
Retrieval Node (<code>agents/retrieval.py</code>)	15
Hybrid Retrieval Pipeline (<code>rag/retrieval.py</code>)	15
Agent Nodes	16
LLM Lifecycle Manager (<code>llm.py</code>)	16
Semantic Memory (<code>memory.py</code>)	16
Network Monitor (<code>network.py</code>)	16
4.3 Data Flow and Processing Pipeline	17
Document Ingestion Flow	17
Query Answering Flow	17
Agent State Machine	17
4.4 Database / Storage Design	17
SQLite Database	18
Vector Store (ChromaDB)	18
BM25 Sparse Index	18
4.5 Interface Design	18
Chapter 5: Implementation	19
5.1 Development Environment and Tools	19
5.2 Key Implementation Details	20
5.2.1 Deterministic Router Design	20
5.2.2 Reciprocal Rank Fusion Implementation	20
5.2.3 Hierarchical Model Strategy	20
5.2.4 Error Boundary Pattern	20
5.2.5 Windows Pipe Buffer Deadlock Prevention	20

5.3 Challenges and Solutions	21
Challenge 1: Context Window Exhaustion on Low-RAM Machines	21
Challenge 2: BM25 and Dense Score Incompatibility	21
Challenge 3: LLM JSON Parsing Failures	21
Chapter 6: Testing and Evaluation	22
6.1 Testing Strategy	22
6.2 Unit and Integration Tests	22
6.3 System Testing	22
6.4 Results and Discussion	23
Chapter 7: Conclusion and Future Work	23
7.1 Summary of Contributions	23
7.2 Limitations	24
7.3 Future Work	24
References	24
Appendix	25

List of Figures

1	Use case diagram for Sage	14
2	High-level system architecture of Sage	15
3	Document ingestion sequence — from file upload to vector and sparse index storage	17
4	Query answering flow through the agent pipeline	17
5	LangGraph agent state machine — control flow for query processing	18
6	UI navigation flow	19

List of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
BM25	Best Matching 25 (probabilistic retrieval function)
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
ER	Entity-Relationship
FTS	Full-Text Search
GGUF	GPT-Generated Unified Format
GPU	Graphics Processing Unit
KU	Knowledge Unit
LLM	Large Language Model
ONNX	Open Neural Network Exchange
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
SPA	Single-Page Application
SSE	Server-Sent Events
VRAM	Video Random Access Memory
WAL	Write-Ahead Logging

Abstract

Academic research and coursework require students to navigate large volumes of study material, retrieve relevant information, and synthesise it into structured outputs such as explanations, quizzes, diagrams, and reports. Existing AI-powered tools typically depend on cloud-hosted large language models, raising concerns around data privacy, recurring costs, and mandatory internet connectivity — constraints that disproportionately affect students in resource-limited environments. This project presents Sage, a desktop application that addresses these limitations by running quantized large language models entirely on-device via llama.cpp, eliminating the need for API keys, cloud infrastructure, or persistent internet access.

Sage employs a multi-agent orchestration architecture built on LangGraph, where a deterministic router dispatches user queries to one of eight specialised agent nodes: explanation with citation formatting, quiz generation and evaluation, Mermaid diagram synthesis with validation, structured study plan generation, multi-source research with PDF export, code diagnosis and repair, extended chain-of-thought reasoning, and general-purpose conversation with long-term semantic memory. The retrieval pipeline combines dense vector search (ChromaDB with BGE-small-en-v1.5 embeddings via FastEmbed) and sparse BM25 retrieval, fused through Reciprocal Rank Fusion, to ground agent responses in ingested curriculum material. A hierarchical model strategy pairs a primary generation model (Qwen3.5-2B on CPU or Qwen3.5-4B on CUDA) with a lightweight utility model (Qwen3.5-0.8B) dedicated to memory extraction, history compression, and auxiliary structured output tasks.

The system is delivered as a native Windows desktop application using pywebview with a React/TypeScript single-page application frontend, backed by a FastAPI server communicating via REST and Server-Sent Events. Informal evaluation demonstrates functional correctness across all agent modes, with response latencies suitable for interactive academic use on consumer hardware. The project contributes a fully integrated, privacy-preserving academic assistant that operates without external dependencies.

Chapter 1: Introduction

1.1 Background

The intersection of natural language processing and education has accelerated considerably with the proliferation of large language models. Tools such as ChatGPT, Gemini, and Claude have demonstrated that conversational AI can assist with explaining concepts, answering questions, and generating structured academic content. However, the dominant deployment model for these systems relies on cloud-hosted inference, which requires persistent internet connectivity, incurs per-query costs, and transmits potentially sensitive academic data, including personal study materials and examination content to third-party servers.

Concurrently, advances in model quantization (notably the GGUF format used by llama.cpp) and the release of capable small language models in the 0.5B–4B parameter range have made it feasible to run inference on consumer-grade hardware. Projects such as Ollama and llama.cpp have lowered the barrier to local LLM deployment, while frameworks such as LangChain and LangGraph provide the orchestration abstractions needed to build multi-step agent workflows around these models. These developments create an opportunity to build academic tools that operate entirely on-device, preserving privacy and eliminating infrastructure costs.

1.2 Problem Statement

Students in academic institutions face several challenges when using AI-powered study tools. First, cloud-based assistants require continuous internet access, which is unreliable or expensive in many institutional and geographic contexts particularly in developing regions where network infrastructure is inconsistent. Second, uploading course materials (lecture slides, textbooks, past papers) to external servers raises legitimate privacy and intellectual property concerns, especially when materials are proprietary or contain unpublished examination content. Third, general-purpose chatbots lack integration with the student’s own curriculum; they cannot retrieve specific information from ingested course materials, generate quizzes grounded in syllabus content, or produce study plans aligned with a particular programme’s structure.

Furthermore, existing offline-capable tools such as PrivateGPT and AnythingLLM focus primarily on document question-answering and lack the specialised academic workflows such as; quiz generation, diagram synthesis, structured study planning, multi-source research with citation tracking, that a student-facing tool requires. There is no integrated, offline-first desktop application that combines retrieval-augmented generation with a suite of tool-augmented academic agent workflows.

1.3 Motivation

The motivation for Sage arises from direct experience with the constraints described above. Students at institutions such as Thal University Bhakkar frequently contend with unreliable internet connectivity, limited budgets for subscription-based AI services, and a lack of specialised academic tools that integrate with their course materials. A locally-deployed assistant that processes queries against ingested lecture content, generates quizzes for self-assessment, renders technical diagrams, and produces structured study plans, all without requiring an internet connection or API key, directly addresses these pain points.

Beyond the immediate practical utility, the project is motivated by the principle that AI-assisted learning tools should be accessible regardless of infrastructure constraints. An offline-first architecture ensures that the tool remains functional during network outages, in examination preparation environments where internet access is restricted, and in contexts where data sovereignty is a concern.

1.4 Project Objectives

The project objectives are as follows:

1. To design and implement a multi-agent orchestration framework using LangGraph that routes user queries to specialised agent nodes based on intent classification.

2. To implement a hybrid retrieval-augmented generation pipeline combining dense vector search (ChromaDB) and sparse retrieval (BM25) with Reciprocal Rank Fusion for grounding responses in ingested curriculum material.
3. To deploy quantized large language models locally via llama.cpp with automatic hardware detection and context window scaling for both CPU and CUDA backends.
4. To implement seven specialised agent workflows: explanation, quiz generation/evaluation, diagram synthesis, study plan generation, multi-source research, code diagnosis, extended reasoning, and general conversation.
5. To develop a long-term semantic memory system that extracts, deduplicates, and retrieves student-specific facts across sessions.
6. To deliver the system as a native Windows desktop application with a React/TypeScript frontend, FastAPI backend, and pywebview shell with custom C++ wrapper.
7. To validate the system through a comprehensive test suite achieving 85%+ code coverage.

1.5 Report Organisation

Chapter 2 reviews the relevant literature on retrieval-augmented generation, local LLM inference, and existing academic assistant tools, identifying the gaps that Sage addresses. Chapter 3 presents the requirements analysis, including stakeholder identification, functional and non-functional requirements, and use case descriptions. Chapter 4 describes the system design in detail, covering the five-layer architecture, component breakdown, data flow pipelines, storage schema, and interface design. Chapter 5 discusses the implementation, highlighting key technical decisions, code excerpts, and challenges encountered during development. Chapter 6 details the testing strategy and presents evaluation results. Chapter 7 concludes the report with a summary of contributions, an honest assessment of limitations, and directions for future work.

Chapter 2: Literature Review

2.1 Overview of the Domain

The domain of AI-assisted academic tools sits at the intersection of information retrieval, natural language generation, and educational technology. The evolution of information retrieval from keyword-based methods (such as TF-IDF and BM25) to dense neural retrieval (using learned embeddings from transformer models) has enabled systems to match queries to documents based on semantic similarity rather than lexical overlap. BM25 remains effective for exact-match and keyword-heavy queries, while dense retrieval excels at capturing paraphrased or conceptually related content. Modern systems increasingly combine both approaches in hybrid configurations.

Large language models, particularly those based on the transformer architecture, have demonstrated broad capabilities in text generation, summarisation, question answering, and structured output production. The release of open-weight models in the sub-10B parameter range including the Qwen, Llama, Phi, and Gemma families, has made it possible to run capable language models on consumer hardware. Quantization techniques, particularly the GGUF format supported by llama.cpp, reduce model memory footprint by representing weights in 4-bit or 8-bit precision with minimal quality degradation, enabling inference on machines with 8–16 GB of RAM.

Retrieval-Augmented Generation (RAG) addresses a fundamental limitation of language models: their knowledge is frozen at the training cutoff and they are prone to hallucination when asked about specific documents they have not seen. RAG pipelines augment the generation process by first retrieving relevant text chunks from an external knowledge base and injecting them into the model’s context window. This grounds the model’s output in verifiable source material, improving factual accuracy for domain-specific queries.

Agent frameworks such as LangGraph extend the RAG paradigm by allowing multi-step workflows where an LLM can route between different processing nodes, invoke external tools (calculators, search engines, code interpreters), and maintain state across steps. This enables more complex academic workflows than simple question-answering, such as generating a quiz, evaluating student responses, or conducting multi-source

research with citation tracking.

2.2 Related Work and Existing Systems

Several existing systems address aspects of the problem space that Sage targets, though none provides the same combination of offline operation, multi-agent workflows, and curriculum-integrated retrieval.

Cloud-based academic assistants such as Elicit and Semantic Scholar provide sophisticated literature search and paper summarisation capabilities. Elicit uses language models to extract claims from papers and synthesise findings across multiple sources. However, both services require internet connectivity and operate exclusively on published academic literature rather than user-provided course materials. They do not support offline use, quiz generation, or diagram synthesis.

General-purpose AI chatbots including ChatGPT, Gemini, and Claude offer conversational assistance with academic topics. These systems benefit from large model sizes and extensive training data, producing high-quality explanations across diverse subjects. However, they cannot be grounded in a student’s specific course materials, they require paid subscriptions for sustained use, and they transmit all queries to external servers. The privacy implications are significant when working with unpublished examination content or proprietary institutional materials.

Local RAG tools such as PrivateGPT and AnythingLLM enable users to run document question-answering pipelines on their own hardware. PrivateGPT integrates with various local LLM backends and vector stores, providing a privacy-preserving alternative to cloud services. AnythingLLM offers a browser-based interface for chatting with documents. Both tools focus on general-purpose document Q&A and lack specialised academic features such as quiz generation, diagram rendering, structured study planning, or agent-based workflows.

2.3 Identified Gaps

The review of existing systems reveals a consistent pattern: tools that provide high-quality AI assistance require cloud connectivity and incur ongoing costs, while tools that operate locally are limited to basic document question-answering without the specialised workflows that academic use demands. No existing system combines all of the following capabilities in a single, offline-first application:

A hybrid retrieval pipeline that merges dense and sparse search results to ground responses in specific course material. A multi-agent architecture where different query types are dispatched to specialised processing nodes rather than handled by a single monolithic prompt. A tool-augmented workflow that integrates sandboxed code execution, diagram rendering, search engines, and document export as first-class agent capabilities. A long-term memory system that learns student-specific facts across sessions and injects them into future interactions. A hierarchical model strategy that offloads auxiliary tasks (memory extraction, history compression) to a lightweight utility model, preserving the primary model’s context window for generation. A native desktop application that manages the full lifecycle of local LLM servers, vector stores, and the frontend, without requiring the user to configure any infrastructure.

Sage was designed to fill this gap by integrating these capabilities into a cohesive system that runs entirely on consumer hardware without external dependencies.

Chapter 3: Requirements Analysis

3.1 Stakeholder Analysis

The primary stakeholders for Sage are undergraduate students in computer science, software engineering, and information technology programmes. These users possess varying levels of technical literacy, from first-year students with basic programming knowledge to final-year students undertaking research projects. Their shared goals include efficient study material retrieval, concept clarification, self-assessment through quizzes, and structured study planning. A key constraint is that many of these users operate in environments

with unreliable or metered internet connectivity, making offline functionality a requirement rather than a convenience.

Secondary stakeholders include academic supervisors and course instructors who may recommend the tool to students, and institution administrators who benefit from a system that keeps student data on-device rather than transmitting it to third-party cloud services. The development team (the project authors) constitutes a further stakeholder category, with interests in maintainability, testability, and extensibility of the codebase.

3.2 Functional Requirements

The functional requirements are grouped by subsystem and derived from the implemented feature set verified in the codebase.

Ingestion Subsystem:

1. The system shall ingest documents in PDF, DOCX, PPTX, Markdown, and plain text formats.
2. The system shall chunk ingested documents into segments of configurable size with configurable overlap.
3. The system shall generate dense vector embeddings for each chunk using `BGE-small-en-v1.5` via `FastEmbed`.
4. The system shall store embeddings and metadata in ChromaDB with support for curriculum collection.
5. The system shall build a BM25 sparse index in parallel with vector storage.

Retrieval Subsystem:

1. The system shall perform hybrid retrieval combining dense cosine-similarity search and BM25 sparse retrieval.
2. The system shall fuse dense and sparse retrieval results using Reciprocal Rank Fusion with a configurable damping constant (default: $k=60$).
3. The system shall support course-scoped retrieval by filtering results to a specific course code.

Agent Subsystem:

1. The system shall classify user intent and route queries to the appropriate agent node via a deterministic router driven by UI dropdown selection.
2. The system shall generate explanations grounded in retrieved Knowledge Units with inline citation tags.
3. The system shall generate quizzes (MCQ, short answer, true/false, code) from retrieved material and evaluate student responses.
4. The system shall generate Mermaid diagram code with syntax validation and SVG rendering, with retry on validation failure.
5. The system shall generate structured day-by-day study plans with prerequisite ordering, spaced repetition, and checkpoints.
6. The system shall conduct multi-source research across arXiv, web (DuckDuckGo), and Wikipedia, producing a cited report with PDF export.
7. The system shall diagnose code errors, apply fixes, execute the fixed code in a sandbox, and explain the repair.
8. The system shall support extended chain-of-thought reasoning with configurable token budgets.
9. The system shall support general-purpose conversation with semantic memory context injection.

Memory Subsystem:

1. The system shall extract structured facts (identity, study, preference) from conversation turns using the utility model.
2. The system shall deduplicate new memories against existing entries using FTS-based similarity search.
3. The system shall inject relevant memories into the system prompt for personalised responses.
4. The system shall compress conversation history when it exceeds a configurable token budget.

Persistence Subsystem:

1. The system shall persist conversation metadata (title, message count, timestamps) in SQLite.

2. The system shall checkpoint LangGraph state per thread using AsyncSqliteSaver.
3. The system shall list, retrieve, and delete conversation sessions via REST API.

Interface Subsystem:

1. The system shall stream agent responses to the frontend via Server-Sent Events.
2. The system shall provide a REST API for chat submission, session management, document upload, and system status.

3.3 Non-Functional Requirements

- **Performance:** The system shall produce initial response tokens within 10 seconds of query submission on a machine with 16 GB RAM and a modern CPU. Batch-mode agents (quiz, diagram, planner) shall complete within 180 seconds.
- **Privacy:** All data processing, including LLM inference, embedding generation, and vector storage, shall occur on the user's local machine. No data shall be transmitted to external servers unless the user explicitly invokes online search tools.
- **Usability:** The system shall present a graphical chat interface requiring no command-line interaction for standard use. Agent mode selection shall be accessible via a single UI dropdown.
- **Portability:** The system shall run on Windows 10 and Windows 11 with Python 3.12. A single NSIS installer shall package all dependencies.
- **Reliability:** Every agent node shall be wrapped with an error boundary that catches exceptions and returns a graceful fallback response, preventing graph-level failures from surfacing as unhandled crashes.
- **Maintainability:** The codebase shall enforce strict import layering (tools → rag → agents) verified by CI, and maintain 80% or greater test coverage.
- **Offline operation:** The system shall function without internet connectivity. Network-dependent features (arXiv, web search, Wikipedia) shall degrade gracefully when offline, with a background monitor detecting connectivity state.

3.4 Use Case Descriptions

3.4.1 Ingest Academic Document

Field	Description
Actor	Student
Precondition	Application is running; document file is accessible on disk
Main Flow	1. Student uploads a document via the file picker. 2. System parses the file (PDF, DOCX, PPTX, MD, TXT). 3. System chunks the text into segments with overlap. 4. System generates embeddings and stores vectors in ChromaDB. 5. System builds/updates the BM25 sparse index. 6. System confirms successful ingestion to the student.
Postcondition	Document chunks are queryable via hybrid retrieval
Exceptions	Unsupported file format; file exceeds maximum size (200 MB); file read error

3.4.2 Ask an Explain Question

Field	Description
Actor	Student

Field	Description
Precondition	Application is running; model is loaded
Main Flow	1. Student types a question and selects “Explain” mode. 2. Router classifies intent as “explain”. 3. Retrieval node executes hybrid search and wraps results as Knowledge Units. 4. Reasoning node generates an explanation grounded in KUs with citation tags. 5. Response generator formats citations. 6. Answer is streamed to the frontend via SSE.
Postcondition	Explanation with inline KU citations is displayed
Exceptions	No relevant chunks found (system answers from general knowledge with disclaimer); LLM timeout

3.4.3 Generate and Take a Quiz

Field	Description
Actor	Student
Precondition	Application is running; course material is ingested
Main Flow	1. Student selects “Quiz” mode and submits a topic. 2. Retrieval node fetches relevant KUs. 3. Quiz node generates 6–10 questions (MCQ, short answer, true/false, code). 4. Student submits answers. 5. Quiz node evaluates responses, identifies misconceptions, and provides a score.
Postcondition	Quiz results with explanations are displayed
Exceptions	Insufficient material for quiz generation; evaluation parsing failure

3.4.4 Conduct Multi-Source Research

Field	Description
Actor	Student
Precondition	Application is running; network is available (for online sources)
Main Flow	1. Student selects “Research” mode and enters a topic. 2. Research node generates a subtopic plan with source-specific queries. 3. System executes parallel searches across arXiv, web, and Wikipedia. 4. Source results are digested into per-subtopic summaries. 5. Report writer synthesises a structured, cited report. 6. System exports the report as PDF via Typst and as Markdown. 7. Artifacts are served for download.
Postcondition	Research report (PDF and Markdown) is available for download
Exceptions	Network unavailable (system reports inability to reach sources); search timeout; report synthesis timeout

3.4.5 Generate a Study Plan

Field	Description
Actor	Student
Precondition	Application is running
Main Flow	1. Student selects “Study Plan” mode and describes the subject and timeline. 2. Planner node analyses the request, extracting topics, difficulty, and prerequisites. 3. System generates a day-by-day schedule with spaced repetition, checkpoints, and self-assessment questions. 4. Schedule is displayed as formatted Markdown.
Postcondition	Structured study roadmap is displayed
Exceptions	Ambiguous request (system asks for clarification); LLM produces invalid JSON (retry with error feedback)

The following use case diagram illustrates the relationships between the student actor and the system’s primary use cases, as described in Figure~1.

Chapter 4: System Design

4.1 System Architecture Overview

Sage is structured as a five-layer architecture, where each layer has a single, well-defined responsibility and communicates with adjacent layers through explicit interfaces. The offline-first constraint is the primary architectural driver: every layer is designed to function without network access, with online features (search tools) treated as optional enhancements that degrade gracefully when unavailable.

The five layers are:

1. **Presentation Layer:** A React 18 / TypeScript single-page application built with Vite and Tailwind CSS, rendered inside a pywebview native window. The frontend communicates with the backend exclusively through HTTP REST endpoints and Server-Sent Events (SSE) for streaming responses.
2. **API Layer:** A FastAPI application served by Uvicorn on a loopback port. This layer exposes REST endpoints for chat submission, session management, document operations, system status, and artifact retrieval. CORS middleware permits same-origin and Vite dev-proxy connections.
3. **Orchestration Layer:** A LangGraph `StateGraph` compiled in `agents/graph.py`. A deterministic router dispatches queries to one of eight specialised agent nodes based on the UI dropdown selection. All state is threaded through a single typed dictionary (`AgentState`), and every node is wrapped with an error boundary.
4. **Inference Layer:** Two concurrent `llama-server` subprocesses managed by `llm.py`. The primary server loads a Qwen3.5 model (2B on CPU or 4B on CUDA) for generation tasks. The utility server loads Qwen3.5-0.8B on CPU for memory extraction, history compression, and auxiliary structured output. Both expose OpenAI-compatible HTTP APIs on dynamically allocated loopback ports.
5. **Storage Layer:** SQLite with WAL mode for conversation metadata, semantic memories, and LangGraph state checkpointing. ChromaDB for vector embeddings with separate collections for curriculum and user uploads. A BM25 pickle index for sparse retrieval.

The following diagram illustrates the high-level architecture, as described in Figure~2.

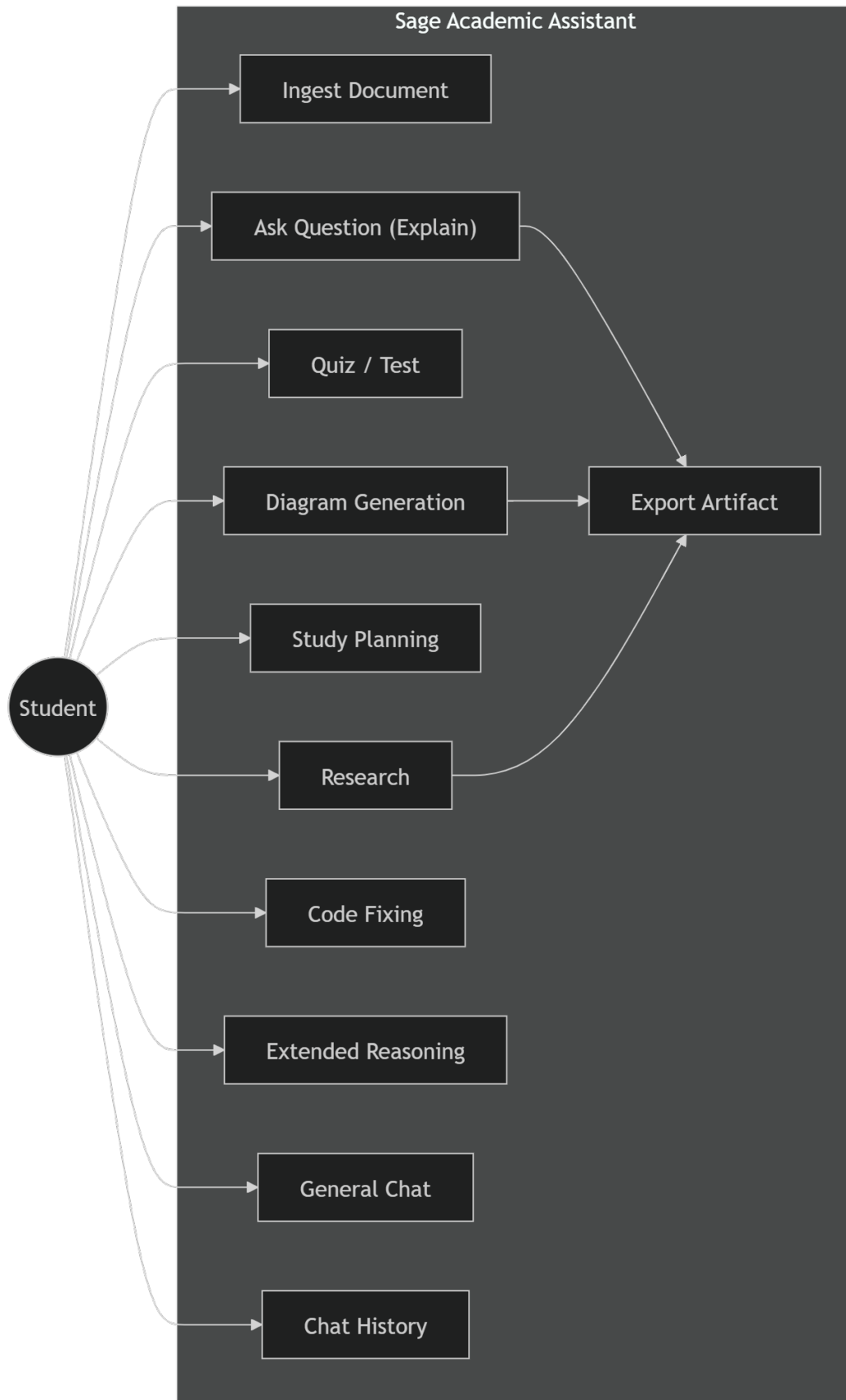


Figure 1: Use case diagram for Sage

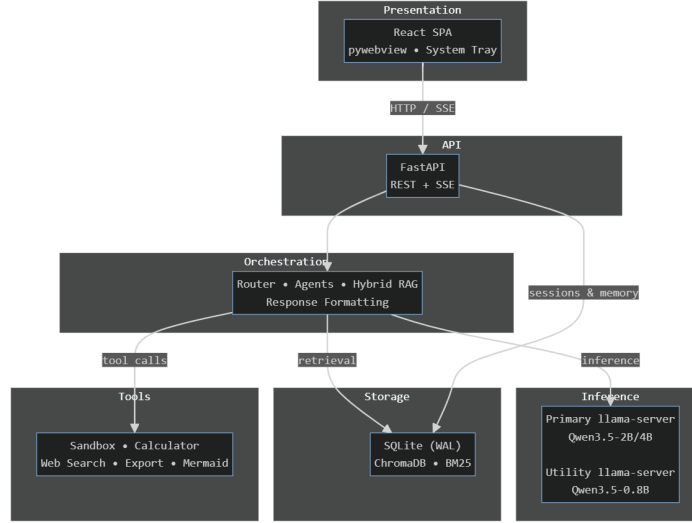


Figure 2: High-level system architecture of Sage

4.2 Component Breakdown

Router (`agents/router.py`)

The router is a fully deterministic component that maps the UI dropdown selection to an intent string. Unlike LLM-based intent classifiers, this approach eliminates classification latency and ensures predictable routing. The `MODE_TO_INTENT` dictionary maps user-facing mode names (e.g., “Quiz Me”, “Study Plan”) to internal intent identifiers (e.g., “quiz”, “roadmap”). The router outputs the intent, the expanded query, and a thinking mode flag.

Retrieval Node (`agents/retrieval.py`)

The retrieval node implements the interface between the agent graph and the hybrid RAG pipeline. It invokes `hybrid_retrieve()` with the user’s query and optional course code filter, receives ranked chunks, and wraps them into a Knowledge Unit (KU) schema. Each KU includes an identifier (KU1, KU2, ...), the chunk text, source metadata, and a confidence level derived from its RRF rank position (high for rank 1, medium for ranks 2–3, low for subsequent ranks). A SHA-256-based cache key prevents redundant retrieval when sequential queries target the same topic.

Hybrid Retrieval Pipeline (`rag/retrieval.py`)

The retrieval pipeline executes dense and sparse searches concurrently using `asyncio.gather`:

- **Dense retrieval:** The query is embedded using BGE-small-en-v1.5 (ONNX via FastEmbed), and cosine similarity search is performed against ChromaDB. The embedding model is loaded once and cached for the process lifetime.
- **Sparse retrieval:** A pre-built BM25 index (pickled with `rank-bm25`) is queried with the tokenized query. Results can be filtered by course code.
- **Fusion:** Both ranked lists are merged using Reciprocal Rank Fusion. For each document, the RRF score is computed as $\sum_{r \in \text{ranks}} \frac{1}{k + \text{rank}_r(d)}$, where k is a damping constant (default: 60).
- **Hydration:** The fused ranking is truncated to `top_k` results, and any chunks present only in the sparse results are fetched from ChromaDB by ID to ensure all returned chunks have complete metadata and text.

Agent Nodes

Each agent node is an async function that receives the shared **AgentState** dict and a bound **ChatOpenAI** instance. All nodes follow a consistent pattern: read relevant state fields, construct a prompt from centralised templates (**prompts.py**), invoke the LLM, parse the response, and return state updates.

- **Reasoning** (**agents/reasoning.py**): Generates chain-of-thought explanations. In “explain” mode, it uses KU-grounded prompts with citation tags; in “thinking” mode, it enables the model’s native thinking capability via **chat_template_kwargs**.
- **Quiz** (**agents/quiz.py**): Generates structured quizzes as JSON (MCQ, short answer, true/false, code types) and evaluates student responses with feedback on misconceptions.
- **Diagram** (**agents/diagram.py**): Uses a two-stage pipeline: first generates a structured diagram description (JSON), then converts it to Mermaid syntax. Validates the output using an external renderer (**mmdr.exe**) and retries with error feedback on syntax failure (up to 3 attempts).
- **Planner** (**agents/planner.py**): Extracts study plan parameters (subject, timeline, topics with prerequisites and difficulty) as JSON, then generates a day-by-day schedule with spaced repetition and checkpoints.
- **Research** (**agents/research.py**): Generates a subtopic plan, executes parallel searches across arXiv, web, and Wikipedia, digests results per subtopic, synthesises a cited report, optionally runs peer review, and exports as PDF/Markdown.
- **Code Fix** (**agents/code_fix.py**): Diagnoses errors as JSON (error type, root cause, affected lines, fix strategy), applies the fix, executes the corrected code in a sandboxed subprocess, and generates an educational explanation.
- **General** (**agents/general.py**): Open-ended conversation with injected semantic memory context and optional tool use.

LLM Lifecycle Manager (**llm.py**)

This module manages the full lifecycle of **llama-server** subprocesses. On startup, it performs GPU detection via **nvidia-smi**, selects the appropriate binary (CUDA or CPU) with DLL integrity checks, resolves context window size from available RAM/VRAM using tiered thresholds, allocates GPU layers based on VRAM capacity, detects hybrid CPU topologies for thread allocation, kills orphaned server processes, spawns the server on a dynamically allocated port, drains stderr in a background thread to prevent Windows pipe buffer deadlocks, polls the **/health** endpoint until ready, and performs a JIT warmup request. Teardown is registered via both **atexit** and **SIGTERM** handlers.

Semantic Memory (**memory.py**)

The memory subsystem provides cross-session personalisation. After each conversation turn, the utility model extracts structured facts categorised as **identity** (name, university, programme), **study** (courses, grades, topics), or **preference** (explanation style, language). New facts are deduplicated against existing entries using FTS5 full-text search; facts exceeding a similarity threshold (default: 0.88) are rejected. On each new query, relevant memories are retrieved by keyword search and injected into the system prompt as a “Student Memory” block.

Network Monitor (**network.py**)

A lightweight async service that performs parallel TCP probes against public endpoints (1.1.1.1:443, 8.8.8.8:53, 1.0.0.1:443) to detect internet connectivity. The monitor runs an initial check on startup and re-checks at configurable intervals, logging state transitions. A **force_offline** configuration option overrides detection for air-gapped environments.

4.3 Data Flow and Processing Pipeline

Document Ingestion Flow

The document ingestion pipeline transforms raw files into queryable vector and sparse index entries, as illustrated in Figure~3.

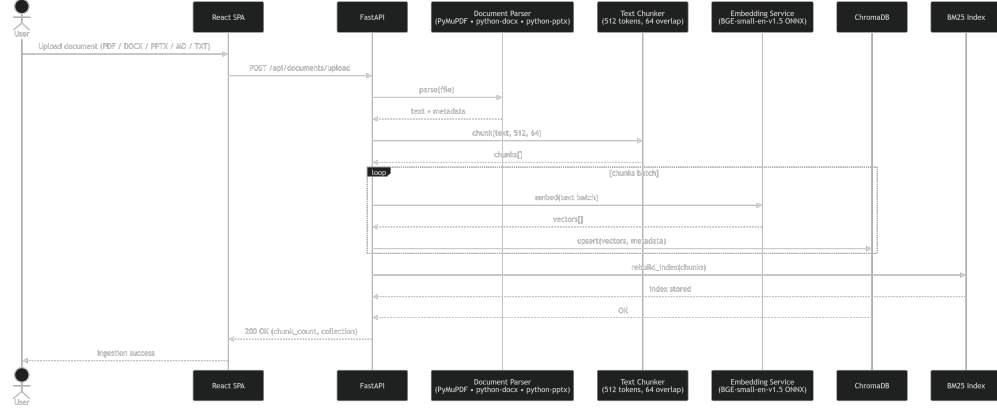


Figure 3: Document ingestion sequence — from file upload to vector and sparse index storage

Query Answering Flow

The query answering pipeline routes user queries through the agent graph, as illustrated in Figure~4.

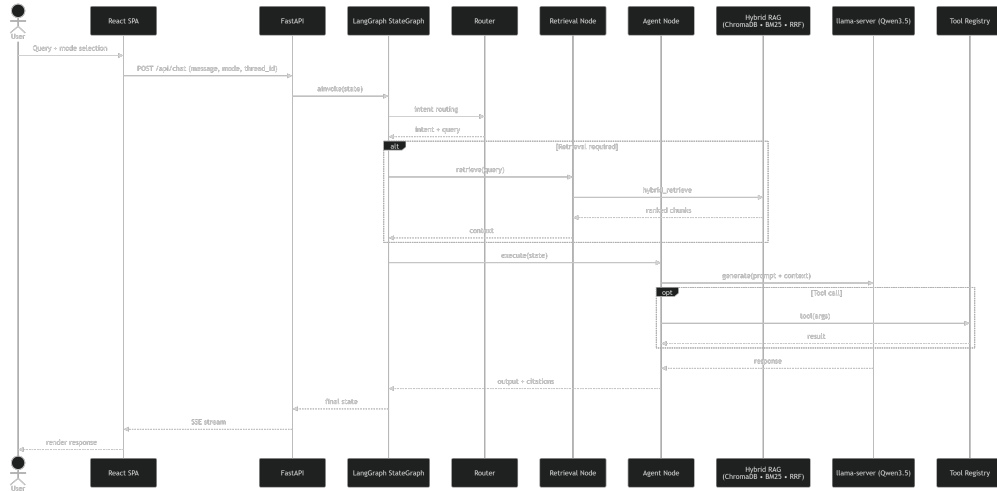


Figure 4: Query answering flow through the agent pipeline

Agent State Machine

The LangGraph state machine governs control flow across agent nodes, as illustrated in Figure~5.

4.4 Database / Storage Design

Sage uses three storage mechanisms, each chosen for its suitability to a specific data type.

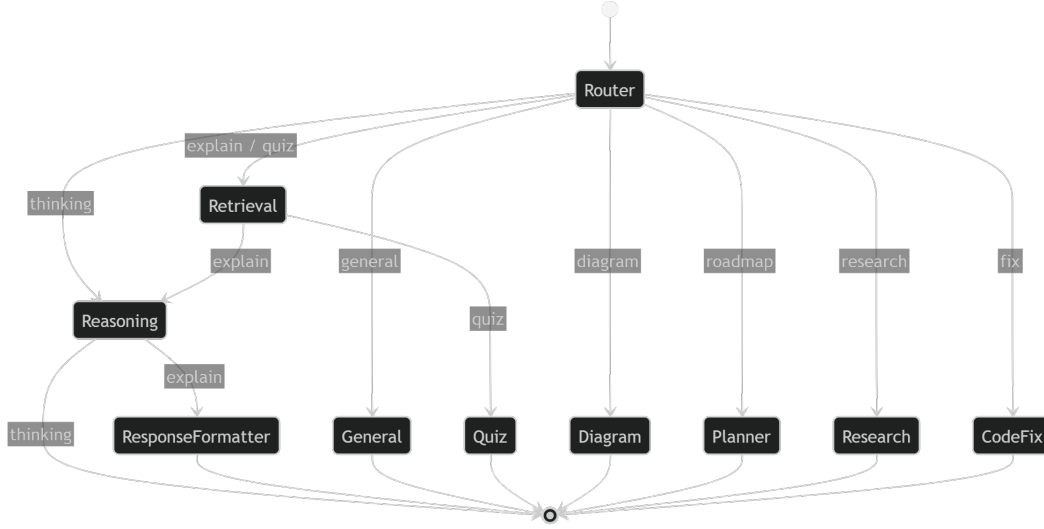


Figure 5: LangGraph agent state machine — control flow for query processing

SQLite Database

The primary relational store uses SQLite with WAL (Write-Ahead Logging) mode enabled for concurrent read/write access. The database schema (version 3) contains the following tables:

- **conversations:** Stores metadata for each chat session (id, title, message_count, last_message, timestamps, archive flag). LangGraph’s `AsyncSqliteSaver` handles the actual message history through its own checkpoint tables in the same database file.
- **memories:** Stores long-term semantic facts about the student (id, content, category, confidence, access_count, timestamps). An FTS5 virtual table (`memories_fts`) with Porter stemming tokenization enables full-text search for deduplication and retrieval. Triggers automatically propagate inserts, updates, and deletes to the FTS index.
- **schema_version:** Version tracking for database migrations.

Vector Store (ChromaDB)

ChromaDB manages vector embeddings on disk at `artifacts/data/databases/vectordb/`. Single collection is maintained for now for pre-ingested course material organised by course code, semester, and programme. Each chunk is stored with metadata fields including `doc_title`, `source_path`, `page_number`, `course_code`, `course_title`, `program_code`, `semester`, and `chunk_index`. Embeddings are 384-dimensional vectors produced by BGE-small-en-v1.5.

BM25 Sparse Index

A pickled `rank-bm25` model stored at `artifacts/data/databases/vectordb/bm25_curriculum.pkl` contains the BM25 object, parallel document IDs, and metadata arrays. The index is rebuilt during ingestion and loaded once at runtime.

4.5 Interface Design

The frontend is a React 18 single-page application written in TypeScript and styled with Tailwind CSS, built with Vite. It is rendered inside a pywebview native window using the Edge WebView2 backend on Windows. The interface follows a single-screen chat layout with the following key elements:

- **Chat View:** The primary view, showing a scrollable message history with Markdown rendering, syntax-highlighted code blocks, LaTeX math rendering, and embedded Mermaid diagrams (rendered

as SVG).

- **Mode Selector:** A dropdown at the top of the chat input area that selects the agent mode (General, Explain, Quiz, Diagram, Study Plan, Research, Fix Code, Thinking).
- **Session Sidebar:** A list of past conversations ordered by recency, with title, last message preview, and timestamps. Sessions can be resumed or deleted.
- **Artifact Downloads:** Generated exports (PDF reports, SVG diagrams, Markdown files) are displayed as downloadable cards.
- **System Status Overlay:** On startup, a loading overlay polls `/api/status` and displays model loading progress. The overlay fades when `model_ready` becomes true.

The following diagram illustrates the navigation flow between UI states, as described in Figure~6.

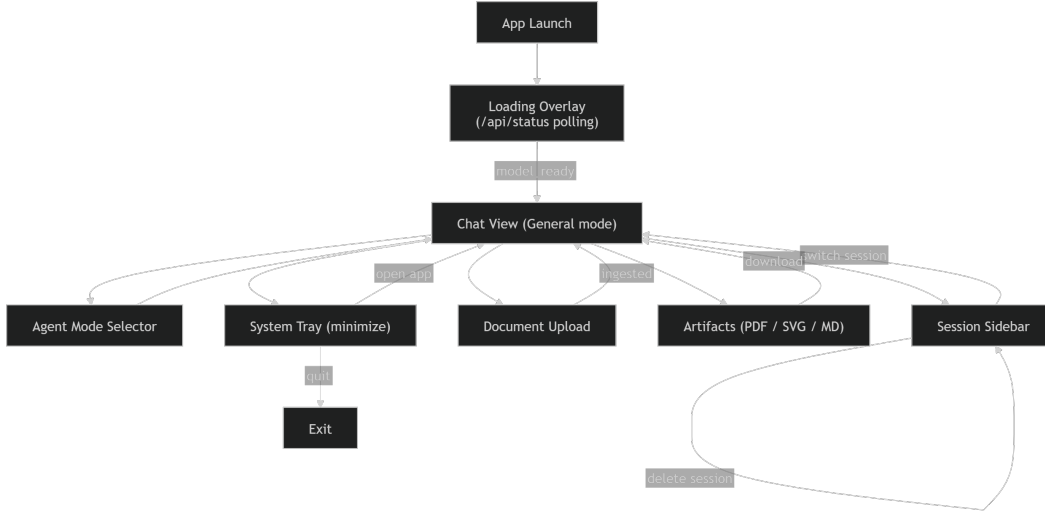


Figure 6: UI navigation flow

Chapter 5: Implementation

5.1 Development Environment and Tools

The following environment and toolchain were used throughout development:

Component	Technology	Version
Operating System	Windows 10/11	—
Language	Python	3.12
Package Manager	uv	Latest
LLM Framework	LangChain, LangGraph	≥ 1.0
LLM Inference	llama.cpp (llama-server)	Latest release
Vector Store	ChromaDB	≥ 0.5
Embeddings	FastEmbed (BGE-small-en-v1.5, ONNX)	$\geq 0.4.2$
Backend	FastAPI, Uvicorn, Pydantic ≥ 2.9	—
Frontend	React 18, TypeScript, Vite, Tailwind CSS	—
Frontend Package Manager	pnpm	≥ 10
Desktop Shell	pywebview, pystray	≥ 6.1
Database	SQLite (via aiosqlite), WAL mode	—
Testing	pytest, pytest-cov, pytest-asyncio	—
Linting	Ruff, Mypy (strict mode)	—
Version Control	Git, GitHub	—

Component	Technology	Version
CI/CD	GitHub Actions	—
Installer	NSIS, PowerShell (<code>build.ps1</code>)	—

5.2 Key Implementation Details

5.2.1 Deterministic Router Design

A deliberate design decision was made to use a fully deterministic router rather than an LLM-based intent classifier. In early prototyping, LLM-based routing introduced 2–5 seconds of latency per query and occasionally misclassified intent, sending quiz requests to the general agent or explain requests to the diagram agent. The deterministic approach maps the UI dropdown selection directly to an intent string via a static dictionary, eliminating both latency and misclassification. This approach trades flexibility (the router cannot handle freeform intent) for reliability and speed. Since the UI provides a constrained set of mode options, the trade-off is acceptable.

5.2.2 Reciprocal Rank Fusion Implementation

The hybrid retrieval pipeline fuses results from dense (ChromaDB) and sparse (BM25) retrievers using Reciprocal Rank Fusion. The damping constant k (default: 60) controls the influence of rank position. Documents appearing in both retrievers receive additive scores, causing them to rank higher in the fused list. This approach was chosen because it requires no tunable weights between retrievers and is robust to differences in score distributions between dense and sparse methods.

5.2.3 Hierarchical Model Strategy

Sage manages two concurrent `llama-server` instances to balance generation quality with resource efficiency. The primary model (Qwen3.5-2B on CPU or Qwen3.5-4B on CUDA) handles all user-facing generation tasks. The utility model (Qwen3.5-0.8B, always CPU) handles background tasks that require structured output but not high generation quality: memory extraction, history compression, conversation title generation, and auxiliary structured extraction for quiz evaluation and diagram description.

This separation was motivated by the observation that dedicating the primary model’s context window and inference capacity to background tasks degraded interactive response latency. On CPU-only systems, the utility model offloads structured extraction steps that would otherwise compete with the primary model for CPU time. On GPU systems, the utility model runs on CPU in parallel with GPU-accelerated primary inference.

5.2.4 Error Boundary Pattern

Every agent node in the LangGraph state graph is wrapped with the `with_error_boundary` decorator. This ensures that an exception in any single node (e.g., an LLM timeout, a JSON parsing failure, or a tool execution error) is caught and converted into a graceful error response rather than crashing the entire graph execution. This pattern was adopted after observing that LLM responses are inherently non-deterministic and that tool calls (particularly sandboxed code execution and network searches) can fail for reasons outside the system’s control. The error boundary ensures the user always receives a response, even if degraded.

5.2.5 Windows Pipe Buffer Deadlock Prevention

A non-obvious challenge is preventing subprocess deadlocks on Windows. When `llama-server` starts, it writes extensive diagnostic output to `stderr` (CUDA device enumeration, layer-by-layer offload progress, quantization details). The Windows default pipe buffer is 64 KB; if the parent process does not read from the pipe, the server blocks on its own write, causing startup to hang indefinitely. The solution is a background drain thread that continuously reads from `stderr` into a bounded rolling buffer.

5.3 Challenges and Solutions

Challenge 1: Context Window Exhaustion on Low-RAM Machines

Challenge: On machines with 8 GB RAM, the primary model’s context window was initially set to a static 8192 tokens, leaving insufficient memory for the model weights, KV cache, and OS overhead. This caused either out-of-memory crashes or extreme swapping.

Root cause: A fixed context window does not account for the wide variation in available RAM across student machines. Model weight memory consumption plus KV cache memory (which scales linearly with context size and quadratically with batch size) must fit within available RAM alongside the Python process and OS.

Solution: The `_resolve_context_size` function in `llm.py` dynamically scales the context window based on available RAM (CPU) or VRAM (GPU) using tiered thresholds. On CPU, context ranges from 3K tokens (under 3 GB available) to 32K tokens (over 5 GB available). The function also accounts for model file size to adjust thresholds for different quantizations.

Outcome: Sage starts reliably on machines with as little as 8 GB total RAM, automatically selecting a context window that fits within available resources.

Challenge 2: BM25 and Dense Score Incompatibility

Challenge: Early attempts to fuse dense and sparse retrieval results by combining raw scores produced inconsistent rankings. BM25 scores are unbounded positive floats, while ChromaDB distance scores are bounded between 0 and 2 (for cosine distance). Normalising scores to a common range introduced tunable parameters that were sensitive to corpus size and query length.

Root cause: Dense and sparse retrieval methods produce scores with fundamentally different distributions and semantics. Linear combination of normalised scores requires careful weight tuning that generalises poorly across query types.

Solution: Reciprocal Rank Fusion was adopted instead of score-based fusion. RRF operates purely on rank positions, eliminating the need for score normalisation or weight tuning. The single hyperparameter (damping constant `k=60`) has been shown to be robust across diverse retrieval settings.

Outcome: Retrieval quality improved for queries that benefit from both semantic (dense) and keyword (sparse) matching, without introducing additional tuning parameters.

Challenge 3: LLM JSON Parsing Failures

Challenge: Several agent nodes (quiz, diagram, planner, research, code fix) require the LLM to produce structured JSON output. Small local models (2B–4B parameters) frequently produce malformed JSON: trailing commas, unquoted keys, truncated arrays, or extraneous text before/after the JSON block.

Root cause: Unlike large cloud models with instruction-following fine-tuning, small quantized models have weaker adherence to strict output format constraints, especially when the prompt is long and the JSON schema is complex.

Solution: A multi-layered parsing strategy was implemented: (1) attempt standard `json.loads()`; (2) strip markdown code fences and retry; (3) use regex to extract the first `{...}` or `[...]` block; (4) on CPU-only builds, delegate structured extraction steps to the utility model, which has a shorter context and produces more predictable output. For diagram generation specifically, a retry loop with error feedback allows the model to correct its own syntax errors.

Outcome: JSON parsing success rates improved significantly. The retry-with-feedback pattern in the diagram agent recovers from most syntax errors within 2 attempts.

Chapter 6: Testing and Evaluation

6.1 Testing Strategy

The testing strategy for Sage employs a combination of unit testing, integration testing, and informal system-level evaluation. The primary testing framework is `pytest` with the `pytest-asyncio` extension for asynchronous code, `pytest-cov` for coverage measurement, and extensive use of mocking to isolate components from the LLM and vector store dependencies that require significant startup time and hardware resources.

The test suite is configured to enforce a minimum coverage threshold of 85% on the `src/sage/` package, with exclusions for modules that are inherently difficult to test in isolation: `__main__.py` (CLI argument parsing and process dispatch), `desktop.py` (pywebview and system tray integration), and `llm.py` (subprocess management with hardware-dependent behaviour).

6.2 Unit and Integration Tests

The test suite comprises 30 test modules in the `tests/` directory, covering the following areas:

Test Module	Coverage Area
<code>test_agents_core.py</code>	Agent node functions (router, retrieval, general, reasoning) with mocked LLM
<code>test_agents_graph.py</code>	Graph compilation and node wiring
<code>test_agents_quiz_extra.py</code>	Quiz generation and evaluation parsing
<code>test_agents_diagram_extra.py</code>	Diagram description and Mermaid generation
<code>test_agents_research.py</code>	Research pipeline (plan, search, digest, report)
<code>test_agents_response.py</code>	Citation formatting
<code>test_agents_retrieval.py</code>	Retrieval node with cache hit/miss scenarios
<code>test_database.py</code>	SQLite operations (conversations, memories, FTS)
<code>test_memory.py</code>	Memory extraction, deduplication, injection
<code>test_tools_logic.py</code>	Calculator, sandbox, export tool logic
<code>test_search_tools.py</code>	arXiv, web, Wikipedia search mocking
<code>test_routers.py</code>	FastAPI endpoint routing and validation
<code>test_routers_chat.py</code>	Chat submission, SSE streaming, error cases
<code>test_utils.py</code>	Utility functions (token estimation, text cleaning)
<code>test_llm_core.py, test_llm_logic.py</code>	GPU detection, binary selection, context scaling
<code>test_desktop.py</code>	Desktop shell lifecycle mocking
<code>test_network.py</code>	Network monitor probe and state transitions
<code>test_app_core.py, test_app_startup.py</code>	Application factory and lifespan
<code>conftest.py</code>	Shared fixtures (mock LLM, mock database, test settings)

Key testing patterns include:

- **LLM mocking:** Agent tests mock the `ChatOpenAI.ainvoke` method to return deterministic responses, enabling assertion on prompt construction and response parsing without requiring a running `llama-server`.
- **Database isolation:** Each test function uses a temporary SQLite database created in-memory, ensuring tests do not interfere with each other or with production data.
- **Async testing:** All async functions are tested with `pytest-asyncio` in auto mode, ensuring correct `asyncio` event loop management.

6.3 System Testing

System-level evaluation was performed through informal, exploratory testing on two hardware configurations:

1. **CPU-only:** Intel Core i5-12400 (6 cores, 12 threads), 8 GB DDR4 RAM, NVMe SSD, Windows 11.
2. **CUDA:** Same CPU with NVIDIA GTX 1660 (6 GB VRAM), enabling GPU-accelerated inference with Qwen3.5-4B.

Test scenarios included:

- **Document ingestion:** Verified successful ingestion of PDF, DOCX, PPTX, Markdown, and plain text files ranging from 1 page to 200+ pages. Confirmed chunks appeared in ChromaDB and BM25 index.
- **Explain mode:** Submitted questions on various CS topics (data structures, algorithms, networking) and verified that responses included KU citation tags when course material was available.
- **Quiz mode:** Generated quizzes on ingested topics and verified JSON structure, question diversity, and evaluation accuracy.
- **Diagram mode:** Requested diagrams for algorithms, architectures, and system designs. Verified Mermaid syntax validity and SVG rendering.
- **Study plan mode:** Requested study plans for different subjects and timelines. Verified prerequisite ordering and day-by-day scheduling.
- **Research mode:** Conducted research queries on technical topics with network access. Verified source search, citation inclusion, and PDF/Markdown export.
- **Offline operation:** Disconnected from the network and verified that all offline agent modes (explain, quiz, diagram, planner, general, thinking, code fix) continued to function correctly. Verified that online tools (search) reported unavailability gracefully.
- **Memory persistence:** Verified that student facts extracted in one session were recalled in subsequent sessions.
- **Session management:** Created, resumed, and deleted sessions. Verified state persistence across application restarts.

6.4 Results and Discussion

Functional completeness: All seven agent modes demonstrated functional correctness during testing. The deterministic router correctly dispatched queries to the intended agent node in every test case. The hybrid retrieval pipeline returned relevant chunks when course material was ingested, and the RRF fusion consistently ranked documents appearing in both retrievers higher than those in only one.

Response quality: Response quality varied with model size and query complexity. The Qwen3.5-4B model (CUDA) produced noticeably more coherent and detailed responses than the 2B model (CPU), particularly for complex explanations and research synthesis. Quiz generation produced well-formed questions with plausible distractors in the majority of cases, though occasional JSON parsing failures required the multi-layered parsing strategy described in Section 5.3.

Performance: On the CPU-only configuration, typical response latencies ranged from 5–15 seconds for short explain-mode queries to 4–5 minutes for research reports requiring multi-source search. On the CUDA configuration, explain-mode latencies dropped to 3–8 seconds. Model loading time (startup) was approximately 15–25 seconds on CPU and 5–15 seconds on GPU.

Memory system: The semantic memory system correctly extracted and persisted facts across sessions. Deduplication via FTS5 effectively prevented storage of semantically identical facts. Memory injection was observed to produce more personalised responses in follow-up sessions (e.g., addressing the student by name, referencing their enrolled courses).

Chapter 7: Conclusion and Future Work

7.1 Summary of Contributions

This project presented Sage, a privacy-preserving, offline-first academic assistant designed specifically for computer science and engineering students. The system addresses the limitations of cloud-dependent AI

tools by running all inference, embedding generation, and retrieval operations locally on consumer hardware.

The key contributions of this project include:

1. **Integrated Academic Workflows:** The implementation of an eight-node agent graph that moves beyond simple question-answering to provide specialised academic capabilities: quiz generation with automated evaluation, Mermaid diagram synthesis with syntax validation, structured study planning, and multi-source research report generation.
2. **Offline-First Architecture:** A robust desktop application architecture that seamlessly manages the lifecycle of local LLM servers (using `llama-server`) and a FastAPI backend, hidden behind a React-based frontend in a native pywebview shell. The system automatically detects hardware capabilities to scale context windows and allocate GPU layers appropriately.
3. **Curriculum-Grounded Retrieval:** A hybrid RAG pipeline that fuses dense vector search (ChromaDB + BGE embeddings) with sparse retrieval (BM25) via Reciprocal Rank Fusion, ensuring that the model’s responses are grounded in the student’s actual study materials.
4. **Semantic Memory System:** A persistent memory architecture that extracts structured facts from conversations, deduplicates them using full-text search, and injects them into future prompts to provide a personalised learning experience without transmitting data off-device.

7.2 Limitations

While the system meets its functional requirements, several limitations remain:

1. **Hardware Constraints:** Although the dynamic context scaling prevents crashes on low-RAM machines, the quality of generation remains bound by the hardware. On an 8 GB RAM CPU-only machine, the 2B model must run with a restricted context window, which limits the volume of retrieved material that can be injected and degrades performance on complex reasoning tasks compared to the 4B model running on a GPU.
2. **Single Document Format Processing:** The ingestion pipeline processes text extracted from PDFs, but it discards images, tables, and complex formatting. As academic materials frequently rely on visual aids, the inability to perform multi-modal retrieval is a significant limitation.
3. **JSON Generation Reliability:** As noted in Section 5.3, small quantized models struggle with strict JSON formatting. While the multi-layered parsing strategy mitigates this issue, parsing failures still occur occasionally, requiring the user to retry the request.

7.3 Future Work

The following enhancements are proposed for future development:

1. **Multi-Modal Ingestion and Vision-Language Models:** Upgrading the document parser to extract images and using a lightweight Vision-Language Model (VLM) such as LLaVA or Moondream to generate text descriptions for index inclusion. This would allow the retrieval system to surface relevant diagrams from course materials.
2. **Speculative Decoding:** Implementing speculative decoding in the `llama-server` backend. By using the existing 0.8B utility model as a draft model for the 4B primary model, generation speed could be significantly increased on hardware where memory bandwidth, rather than compute, is the bottleneck.
3. **Formal User Studies:** Conducting structured user studies with students to measure the impact of the tool on study efficiency and learning outcomes.
4. **Graph Database for Prerequisites:** Replacing the purely LLM-driven prerequisite logic in the Study Planner with a lightweight graph database that maps the dependency structures of CS curricula, enabling more rigorous and mathematically sound study scheduling.

References

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Stoyanov, V. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*,

33, 9459-9474.

- [2] Gerganov, G. (2023). *llama.cpp: Port of Facebook's LLaMA model in C/C++*. GitHub repository. <https://github.com/ggerganov/llama.cpp>
- [3] Cormack, G. V., Clarke, C. L., & Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval* (pp. 258-259).
- [4] Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389.
- [5] Qwen Team (2024). *Qwen2.5: A Party of Foundation Models!* <https://qwenlm.github.io/blog/qwen2.5/>
- [6] LangChain AI (2024). *LangGraph Documentation*. <https://langchain-ai.github.io/langgraph/>
- [7] BAAI (2023). *BGE: MTEB Leaderboard Top-1 Embedding Model*. <https://github.com/FlagOpen/FlagEmbedding>

Appendix

To install and run Sage on a Windows machine:

1. Download the latest **Sage-Installer-vX.X.X.exe** release.
2. Run the installer and follow the prompts. The installer will extract the Python environment, Node.js frontend, and required model binaries.
3. Launch Sage from the Desktop shortcut or Start Menu.
4. On first launch, the system will detect your hardware (CPU or NVIDIA GPU) and load the appropriate model. This process takes 10-20 seconds.
5. Select an agent mode from the dropdown (e.g., “Explain”, “Quiz Me”) and begin chatting.